SENTHIL KUMAR, APPLIED AI ENGINEER

senthilkumar.m1901@gmail.com • +91-984-186-9609 • GitHub • LinkedIn • Blog • Medium • Chennai, India

- 🧢 🪖 The hats I wear as an Applied AI Engineer:
  - An AWS Cloud Engineer: Architect & deploy backend AI systems in AWS
  - A Generative AI Engineer: Build agentic apps and LLM-powered products
  - An ML Engineer: Leverage self-hosted deep learning for inferencing in NLP applications
- ✍️ Passionate learner and technical writer: Writes at senthilkumarm1901.github.io and on Medium

WORK EXPERIENCE

Lead ML Engineer, Toyota Connected India **Jul 2022 – Present**

- Build production-grade LLM-workflows, RAGs and Agentic GenAI applications
- Design event-driven serverless systems on AWS (Lambda, API Gateway, DynamoDB, etc.).
- Lead a group of BE & FE engineers in AI & cloud projects
- Have published 4 articles in TCIN medium channel
- Have delivered a Talk on Serverless Python App in PyCon2023

Lead Data Scientist, Ford, Analytics Team **May 2018 – Jun 2022**

- A hands-on **data science developer** who co-developed with NLP experts from the US team
- Leveraged open source models for inferencing in dockerized NLP applications

Assistant Manager, LatentView Analytics (LV) **Apr 2014 – Apr 2018**

- Roles I played: **NLP Data Scientist**, **Project Delivery Manager**
- Responsible for project scoping and accountable for the delivery of Social Media Analytics projects of 8+ members

Lead Analyst, Beroe Inc **Jul 2010 – Dec 2013**

- Produce Market Research reports on how to procure indirect spend categories

PERFORMANCE AWARDS

- **Toyota** – Employee of the Month (Sep 2023); Hackathon Team Winner (Mar 2023); 2nd place, AWS India Manufacturing Teams Hackathon 2024; Winner, DriveDesign'25 & AWS Game Day Security Event '26
- **Ford** – Asia-Pacific Recognition Award (May 2019)
- **LatentView** – Encore Award (Jul–Sep 2016 Quarter)
- **Beroe** – Knowledge Contributor Award (twice, 2013)

EDUCATION

- Bachelors in Engg - Electronics - 8.6 CGPA | Madras Institute of Technology, 2006 - 2010
- State Topper in 2006 TN Engineering Entrance Exam (Physical Science 50/50); 12th Grade 95%, 10th Grade 92%

TECHNICAL UPSKILLING

- A 3-Course Specialization on AWS GenAI & Bedrock, Coursera-AWS, Jan'26
- Modern AI Agents, O'Reilly, Aug'25 | Building Apps with Anthropic's MCP, DeepLearning.AI, Jun'25
- A Proctored Exam on Git, Linux Foundation, Jan'25
- DevOps on AWS Specialization(4 courses), Coursera-AWS, Oct'24
- Generative AI with LLMs, Coursera-Deeplearning.ai, Nov'23
- DeepLearning Specialization(5 courses), Coursera-Deeplearning.ai, Dec'18 - May'19

Skills

- **Languages**: Python, SQL(basics), Markdown, Bash/Shell Scripting
- **Python Libraries**: Boto3, LangGraph, Strands, FastAPI, PyTorch, Transformers, SpaCy,
- **Tools**: Git, AWS CLI/Serverless/Bedrock, Docker, Python Env Management, Agentic Enginering

- 👀 **Curious what have I been cooking lately?** 🔗 LinkedIn Featured Section

KEY PROJECTS MORE DETAILS

Learn by Blogging | 35+ Articles | 2020 - Present

- Have been passionately writing about coding, GenAI, Cloud and ML since 2020 here and in Medium
- Example blog posts: LangGraph, Strands, and Hermes Comparison; Single LLMs to Agentic AI: GenAI Evolution Explained; Git: TimeTravel Masterclass; Leveraging LLMs in the Cloud

---

xAgents — Enterprise Agentic AI Platform | 2026 | Ongoing

- **Impact**: Building a desktop-first Agentic AI Platform letting non-developers create, govern, and operate AI agents safely — scaling to 100s–1000s of files per run.
- **Architecture**: React UI → Tauri IPC → Rust BE; Agent Loop inspired by hermes agent loop (refer hermes doc here)
- **Tech Stack**: Rust | Tauri 2 | React + TypeScript + Vite | SQLite | Bedrock · Azure OpenAI · Ollama

---

Restricted-Output Conversational AI Agent | Customer Service Automation | 2025–26

- **Impact**: Built a reliable, controlled, explainable customer-service agent validated across three concurrent use cases (booking, recall appointment, new car sales) on one orchestration backbone.
- **Architecture**: Designed a two-layer FSM system — YAML-declared FSM decides *what*, an execution graph does *how* — with a single `context.py` as runtime source of truth.
- **Tech Stack**: Python | YAML-declared FSM | Node.js + Socket.IO | React + Vite | MongoDB | Azure OpenAI (GPT-4o)

---

xPolicyBot — Agentic RAG Chatbot for Enterprise | HR · IT · InfoSec | 2025

- **Impact**: Delivered a role-aware, multilingual policy assistant with *every chunk source-cited* — the non-negotiable trust contract for policy guidance.
- **Architecture**: Architected an agentic RAG system with planning, reasoning, and reflection loops, extended with MCP-based tool execution so the agent can query structured systems, not just documents.
- **Complexity / Trade-offs**: Moved from Bedrock managed agent orchestration (`agent-runtime`) to self-orchestrated (`bedrock agentcore`) RAG for finer control over planning, prompts, and cost.
- **Tech Stack**: AWS Bedrock (agent runtime, agentcore, Claude Opus) | AWS OpenSearch (vector store) | MCP | OpenAPI / Action Groups | Amazon RDS

---

xFlow — GenAI Workflow Platform | Internal Productivity Platform | 2024

- **Impact**: Delivered a role-aware, multilingual policy assistant with *every chunk source-cited* — the non-negotiable trust provided in the answers
- **Architecture**: Architected an agentic RAG system with planning, reasoning, and reflection loops, extended with MCP-based tool execution so the agent can query structured systems, not just documents.
- **Complexity / Trade-offs**: Bridged OpenAPI-based tool schemas with the MCP JSON tool schema via an adapter layer — preserving a clean control-plane / data-plane boundary; later moved from managed agent orchestration to self-orchestrated RAG for finer control over planning, prompts, and cost.
- **Tech Stack**: AWS Bedrock (agent runtime, Claude Opus) | AWS OpenSearch (vector store) | MCP | OpenAPI / Action Groups | Amazon RDS

Call Center Analytics Dashboard | Contact Center Intelligence | 2024

- **Impact**: Architected a speech-to-insight pipeline turning raw contact-center audio into agent-level performance metrics on a dashboard — replacing manual call-listening sampling with continuous automated analysis.
- **Architecture**: Designed a staged pipeline — audio → diarized turn-level transcript → LLM-driven metric extraction → persisted insights → dashboard — with each stage independently swappable.
- **Complexity / Trade-offs**: Started with an open-source PoC for diarization + STT and metric extraction; given low call volume, migrated to serverless managed APIs for both stages — cheaper to run, zero infra to maintain, and faster to evolve than self-hosted models.
- **Tech Stack**: AWS Transcribe (diarization + STT) | LLM API (metric extraction) | AWS (Lambda · S3) | Dashboard UI
